Internet of Things

Vipul Singh Negi

Department of Computer Science Engineering

National Institute of Technology Rourkela-769008

Outline

- Definition of IoT.
- History of IoT.
- Arduino.
- Basic Arduino Programming.
- Demos.

Definition of Internet of Things

- IoT refers to the interconnection via the Internet of computing devices embedded in everyday objects, enabling them to send and receive data.
- The scope of IoT is not just limited to getting the devices connected or networked, but its more about the exchange of meaningful information from one device to another device for the accurate interpretation of raw data.
- IoT is not a single technology, but a combination of technologies and domain knowledge. So IoT is not owned by one engineering branch. It is a reality when multiple domains come together.

History of IoT

- In the year 1999 A visionary technologist, Kevin Ashton was giving a presentation for Procter & Gamble where he described IoT as a technology that connected several devices with the help of RFID tags for the supply chain management.
- He specifically used the word “internet” in the title of his presentation in order to draw the audience’s attention since the internet was just becoming a big deal that time.
- While his idea of RFID-based device connectivity differs from today’s IP-based IoT, Ashton’s breakthrough played an essential role in the internet of things history and technological development overall.
- Therefore Kevin Ashton coined the term “the internet of things” and is popularly known as the father of IoT.

Cond.

- The concept of connected devices itself dates back to 1832 when the first electromagnetic telegraph was designed. The telegraph enabled direct communication between two machines through the transfer of electrical signals. However, the true history of enterprise IoT started with the invention of the internet—a very essential component—in the late 1960s, which then developed rapidly over the next decades.

The 1980S

- This might be hard to believe, but the first connected device was a Coca-Cola vending machine situated at the Carnegie Melon University and operated by local programmers. They integrated micro-switches into the machine and used an early form of the internet to see if the cooling device was keeping the drinks cold enough and if there were available Coke cans. This invention fostered further studies in the field and the development of interconnected machines all over the world.

The 1990s

- In 1990, John Romkey connected a toaster to the internet for the very first time with a TCP/IP protocol. One year later, University of Cambridge scientists came up with the idea to use the first web camera prototype to monitor the amount of coffee available in their local computer lab's coffee pot. They programmed the webcam to take pictures of the coffee pot three times per minute, then send the images to local computers, thus allowing everyone to see if there was coffee available.

Contd.

- The year 1999 was easily one of the most significant for IoT history, as Kevin Ashton coined the term “the internet of things.” A visionary technologist, Ashton was giving a presentation for Procter & Gamble where he described IoT as a technology that connected several devices with the help of RFID tags for the supply chain management. He specifically used the word “internet” in the title of his presentation in order to draw the audience’s attention since the internet was just becoming a big deal that time. While his idea of RFID-based device connectivity differs from today’s IP-based IoT, Ashton’s breakthrough played an essential role in the internet of thing’s history and technological development overall.

The 2000s

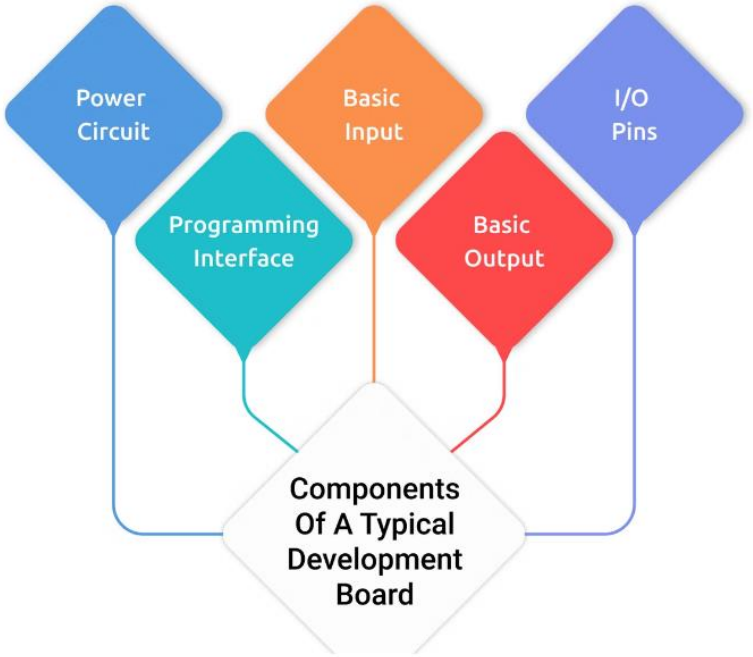
- At the beginning of the 21st century, the term “internet of things” came into widespread use by the media, with outlets like The Guardian, Forbes, and the Boston Globe making mention of it. Interest in IoT technology was steadily increasing, which led to the 1st International Conference on the Internet of Things held in Switzerland in 2008, where participants from 23 countries discussed RFID, short-range wireless communications, and sensor networks.
- Moreover, several major developments fostered the IoT evolution. One was a refrigerator connected to the internet that was introduced by LG Electronics in 2000, allowing its users to shop online and make video calls. Another essential development was a small rabbit-shaped robot named Nabaztag created in 2005 that was capable of telling the latest news, weather forecast, and stock market changes.

The 2010s

- The IoT boom was supported by its addition to the Gartner Hype Cycle for emerging technologies in 2011.
- In the same year, IPv6—a network layer protocol that is central to IoT—was launched publicly.
- Since then, interconnected devices have become widespread and commonplace in our everyday lives. Global tech giants like Apple, Samsung, Google, Cisco, and General Motors are focusing their efforts on the production of IoT sensors and devices—from interconnected thermostats and smart glasses to self-driving cars. IoT has found its way into almost every industry: manufacturing, healthcare, transportation, oil & energy, agriculture, retail, and many more. This dramatic shift has us convinced that the IoT revolution is right here, right now.

The Boards

- A development board is simply a printed circuit board with circuitry and hardware designed to assist experiments with a specific microcontroller. Let us break it down.

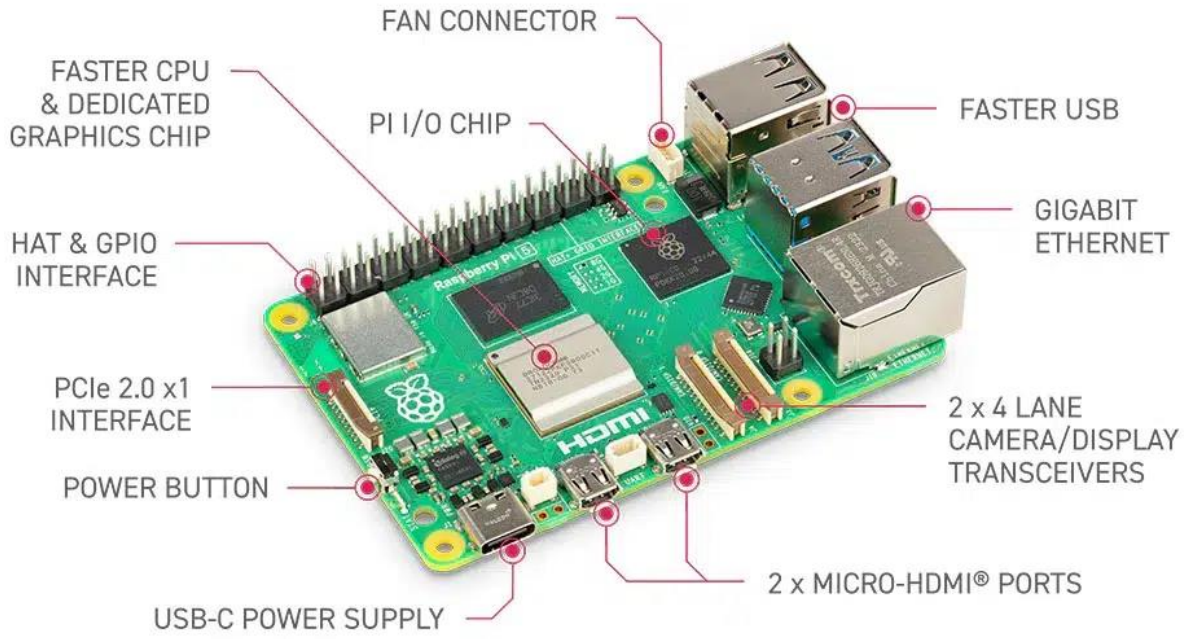


Key features to look for in an IoT board

- Connectivity superiority.
- Scalability options.
- Peripheral support.
- Processing power.
- Board memory.
- Wireless capabilities.

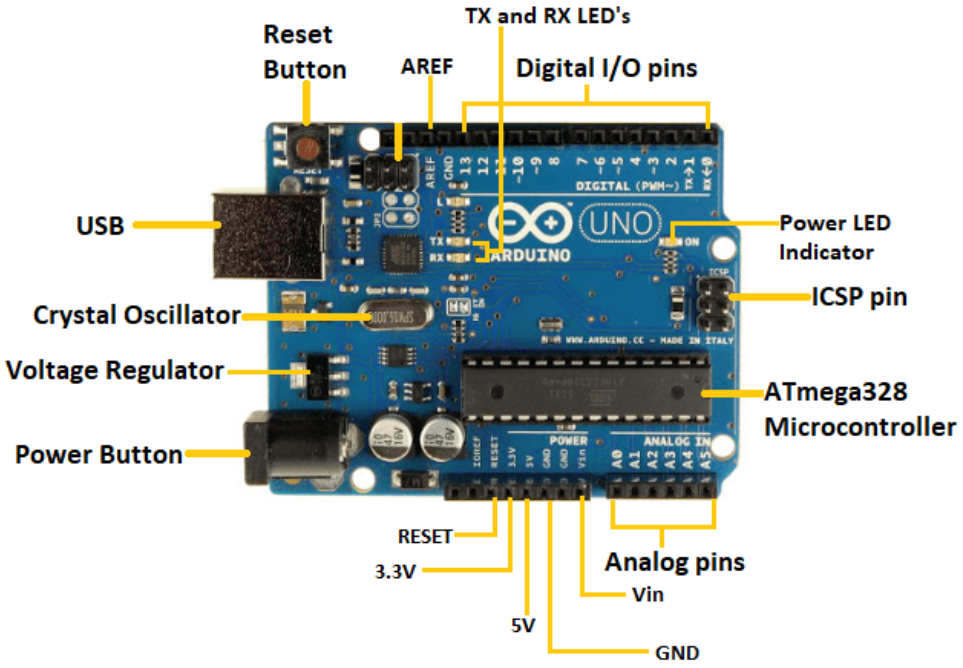
Raspberry Pi 5

- Broadcom BCM2712, 2.4GHz Quad core 64-bit ARM Cortex-A76 CPU.
- 1GB, 2GB, 4GB, 8GB or 16GB LPDDR4X-4267 SDRAM (depending on model)
- 2.4 GHz and 5.0 GHz IEEE 802.11ac wireless, Bluetooth 5.0, BLE.
- Gigabit Ethernet (with PoE+).
- 2 USB 3.0 ports; 2 USB 2.0 ports.



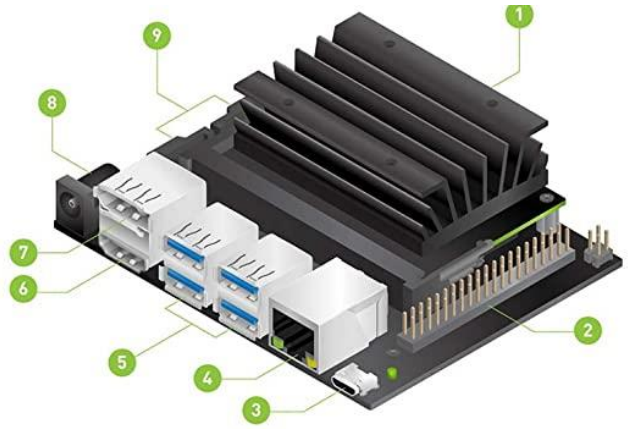
Arduino Uno

- Microcontroller: Microchip ATmega328P.
- Operating Voltage: 5 Volts.
- Input Voltage: 7 to 20 Volts.
- Digital I/O Pins: 14 (of which 6 can provide PWM output)



Jetson Nano

- GPU: 128-core NVIDIA Maxwell™ architecture-based GPU.
- CPU: Quad-core ARM® A57.
- Camera: MIPI CSI-2 DPHY lanes, 12x (Module) and 1x (Developer Kit)
- Memory: 4 GB 64-bit LPDDR4; 25.6 gigabytes/second.
- Connectivity: Gigabit Ethernet.
- OS Support: Linux for Tegra®



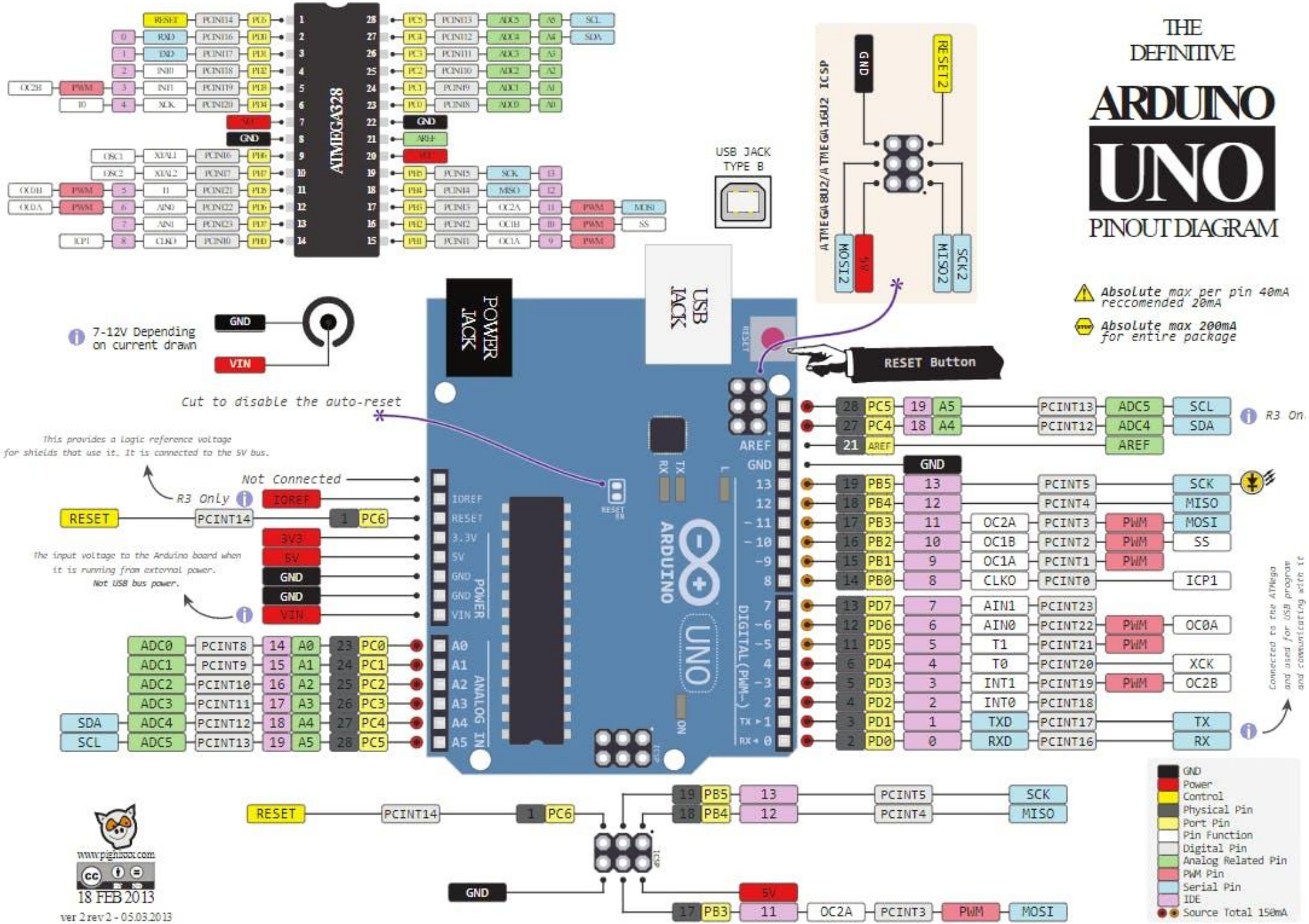
1.Micro SD card slot: insert a 16GB or larger TF card for main storage and writing system image
2.40-pin expansion header
3.Micro USB port: for 5V power input or for USB data transmission
4.Gigabit Ethernet port: 10/100/1000Base-T auto-negotiation
5.4x USB 3.0 port
6.HDMI output port
7.DisplayPort connector
8.DC jack: for 5V power input
9.2x MIPI CSI camera connector

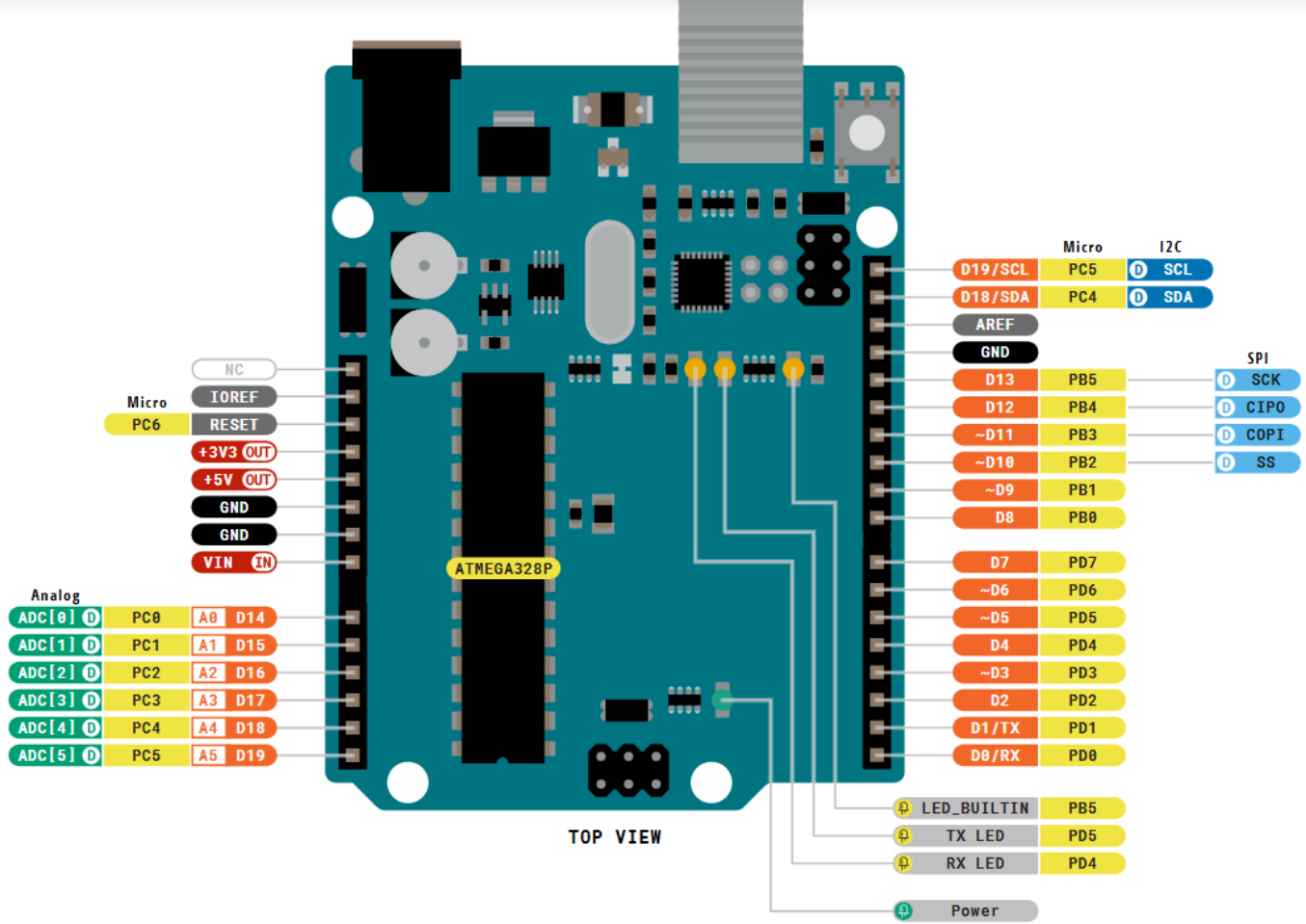
WHAT IS ARDUINO?

- Arduino is an open-source electronics platform based on easy-to-use hardware and software.
- Arduino boards are able to read inputs - light on a sensor, a finger on a button, or a Twitter message - and turn it into an output - activating a motor, turning on an LED, publishing something online.

WHY ARDUINO?

- Thanks to its simple and accessible user experience, Arduino has been used in thousands of different projects and applications.
- The Arduino software is easy-to-use for beginners, yet flexible enough for advanced users. It runs on Mac, Windows, and Linux.


THE DEFINITIVE
ARDUINO UNO
PINOUT DIAGRAM
ATMEGA328
USB JACK TYPE B
ATMEGA8U2/ATMEGA16U2 ICSP
GND
RESET2
MOSI2
5V
MISO2
SCK2
Absolute max per pin 40mA reccomended 20mA
Absolute max 200mA for entire package
POWER JACK
USB JACK
7-12V Depending on current drawn
GND
VIN
RESET Button
Cut to disable the auto-reset
This provides a logic reference voltage for shields that use it. It is connected to the 5V bus.
Not Connected
R3 Only
IOREF
RESET
PCINT14
PC6
3V3
5V
GND
GND
VIN
The input voltage to the Arduino board when it is running from external power. Not USB bus power.
POWER
ANALOG IN
DIGITAL(PWM~)
ARDUINO
UNO
AREF
GND
ADC0 PCINT8 14 A0 23 PC0
ADC1 PCINT9 15 A1 24 PC1
ADC2 PCINT10 16 A2 25 PC2
ADC3 PCINT11 17 A3 26 PC3
SDA ADC4 PCINT12 18 A4 27 PC4
SCL ADC5 PCINT13 19 A5 28 PC5
PCINT13 ADC5 SCL
PCINT12 ADC4 SDA
AREF
R3 On
PB5 13 PCINT5 SCK
PB4 12 PCINT4 MISO
PB3 11 OC2A PCINT3 PWM MOSI
PB2 10 OC1B PCINT2 PWM SS
PB1 9 OC1A PCINT1 PWM
PB0 8 CLKO PCINT0 ICP1
PD7 7 AIN1 PCINT23
PD6 6 AIN0 PCINT22 PWM OC0A
PD5 5 T1 PCINT21 PWM
PD4 4 T0 PCINT20 XCK
PD3 3 INT1 PCINT19 PWM OC2B
PD2 2 INT0 PCINT18
PD1 1 TXD PCINT17 TX
PD0 0 RXD PCINT16 RX
Connected to the ATMega and used for USB program and communicating with it
RESET PCINT14 1 PC6
19 PB5 13 PCINT5 SCK
18 PB4 12 PCINT4 MISO
ICSP
GND
5V
17 PB3 11 OC2A PCINT3 PWM MOSI
GND
Power
Control
Physical Pin
Port Pin
Pin Function
Digital Pin
Analog Related Pin
PWM Pin
Serial Pin
IDE
Source Total 150mA
www.pighixxx.com
18 FEB 2013
ver 2 rev 2 - 05.03.2013



Legend:

- Power
- Ground
- Power Input
- Power Output
- GPIO Digital External
- Analog External
- Main Part
- Secondary Part
- Internal Component
- Other Pins (Reset, System Control, Debugging)
- I2C
- SPI
- UART/USART
- Other SERIAL Communication
- Analog
- PWM/Timer
- Default
- LED
- RGB LED
- Other

MAXIMUM current per I/O pin is 20mA

MAXIMUM current per +3.3V pin is 50mA

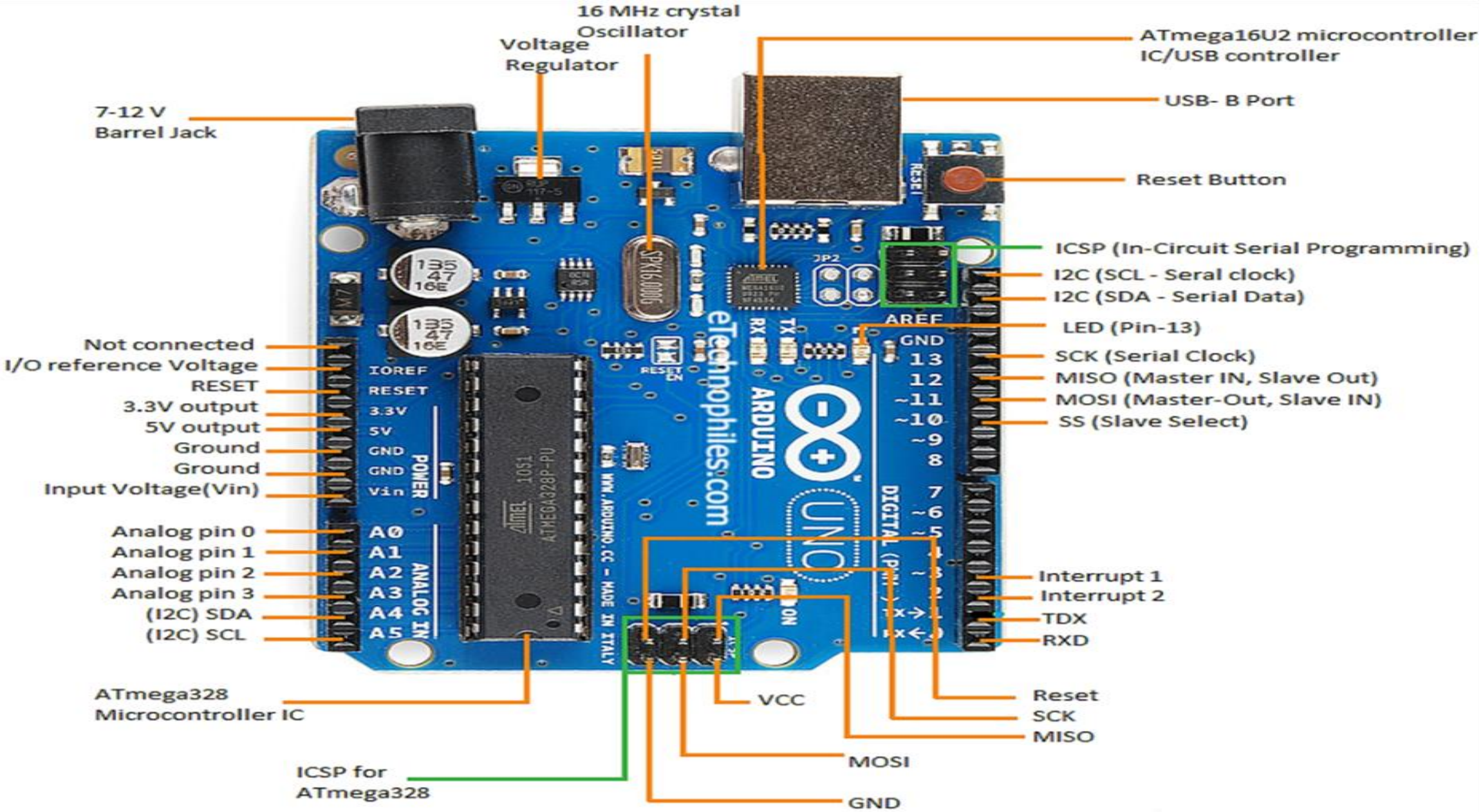
VIN 6-20V input to the board

CIPO/COPI have previously been referred to as MISO/MOSI

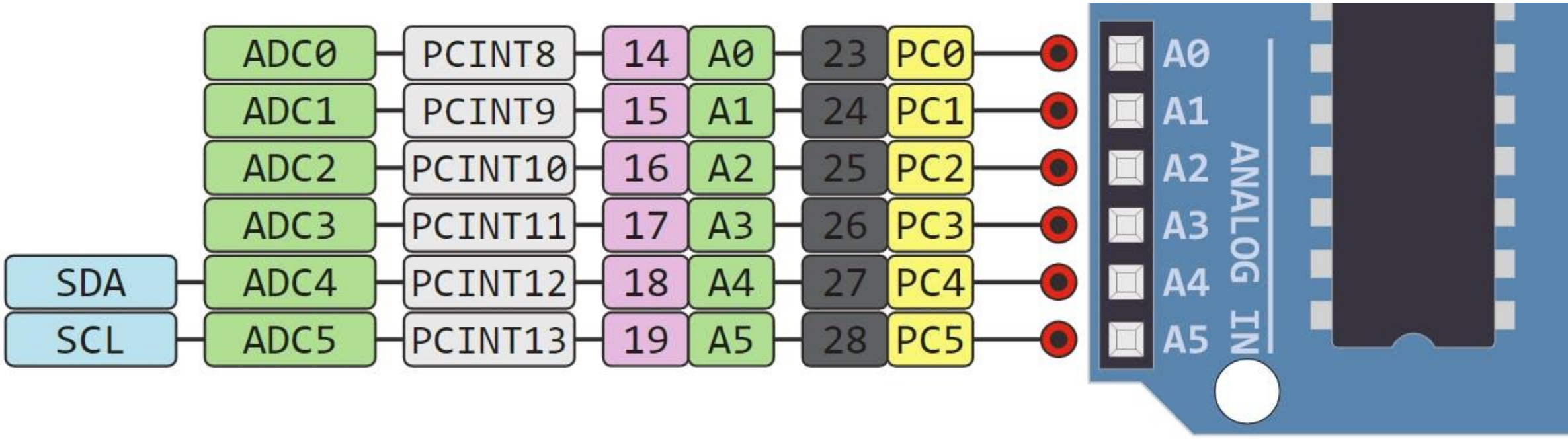
ARDUINO UNO R3 BOARD

SKU code: A000066
Full Pinout - Page 1 of 5
Last update: 6 Oct, 2022

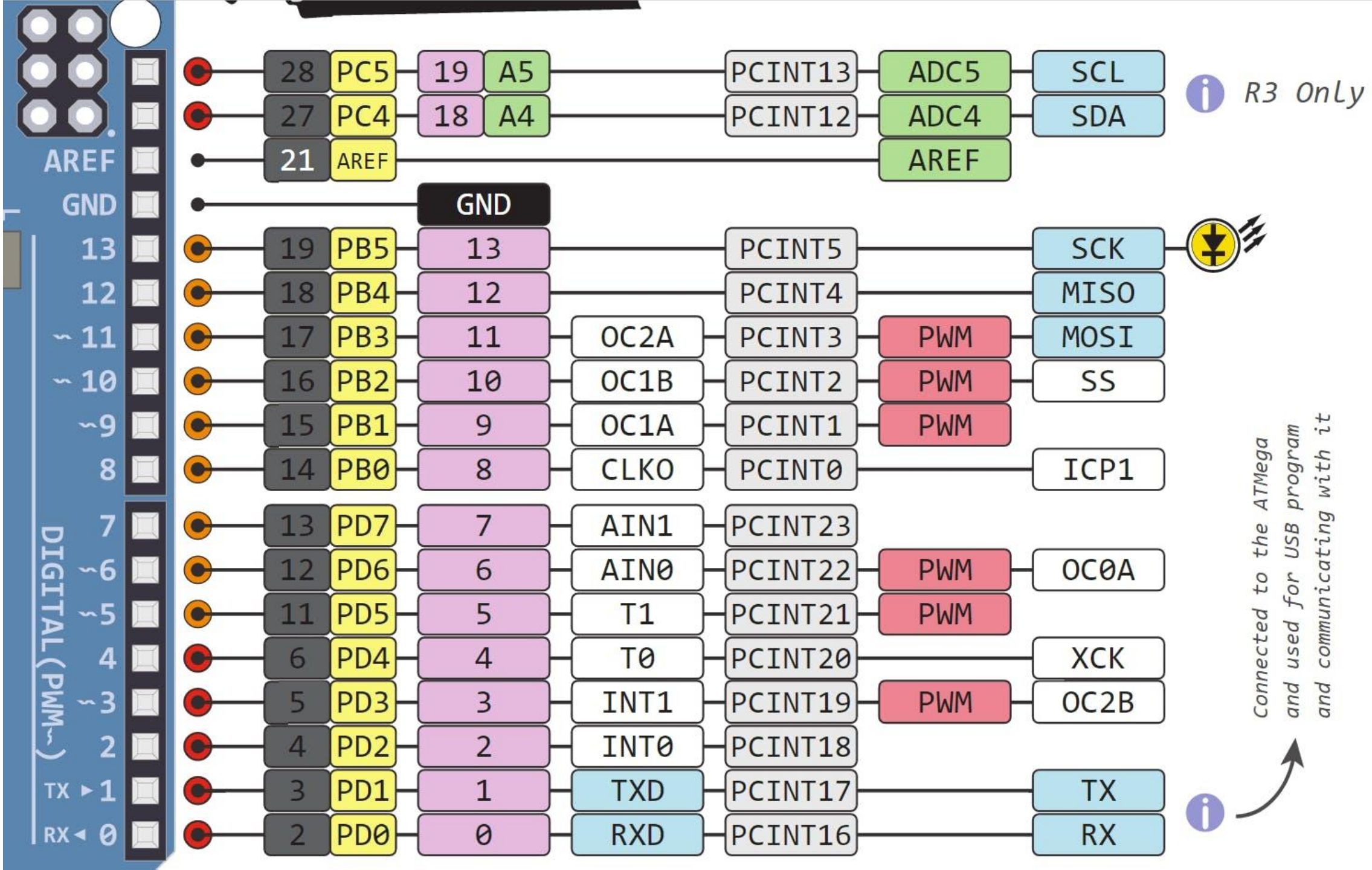
DOCS.ARDUINO.CC



A basic Arduino Uno board


ADC0
PCINT8
14
A0
23
PC0
ADC1
PCINT9
15
A1
24
PC1
ADC2
PCINT10
16
A2
25
PC2
ADC3
PCINT11
17
A3
26
PC3
SDA
ADC4
PCINT12
18
A4
27
PC4
SCL
ADC5
PCINT13
19
A5
28
PC5
A0
A1
A2
A3
A4
A5
ANALOG IN

- ADC stands for Analog to Digital Converter. ADC is an electronic circuit used to convert analog signals into digital signals. This digital representation of analog signals allows the processor (which is a digital device) to measure the analog signal and use it through its operation.
- Arduino Pins A0-A5 are capable of reading analog voltages. On Arduino the ADC has 10-bit resolution, meaning it can represent analog voltage by 1,024 digital levels. The ADC converts voltage into bits which the microprocessor can understand.


AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX →1
RX ←0
DIGITAL(PWM~)
28 PC5 19 A5 PCINT13 ADC5 SCL
27 PC4 18 A4 PCINT12 ADC4 SDA
21 AREF AREF
GND
19 PB5 13 PCINT5 SCK
18 PB4 12 PCINT4 MISO
17 PB3 11 OC2A PCINT3 PWM MOSI
16 PB2 10 OC1B PCINT2 PWM SS
15 PB1 9 OC1A PCINT1 PWM
14 PB0 8 CLKO PCINT0 ICP1
13 PD7 7 AIN1 PCINT23
12 PD6 6 AIN0 PCINT22 PWM OC0A
11 PD5 5 T1 PCINT21 PWM
6 PD4 4 T0 PCINT20 XCK
5 PD3 3 INT1 PCINT19 PWM OC2B
4 PD2 2 INT0 PCINT18
3 PD1 1 TXD PCINT17 TX
2 PD0 0 RXD PCINT16 RX
R3 Only
Connected to the ATMega
and used for USB program
and communicating with it

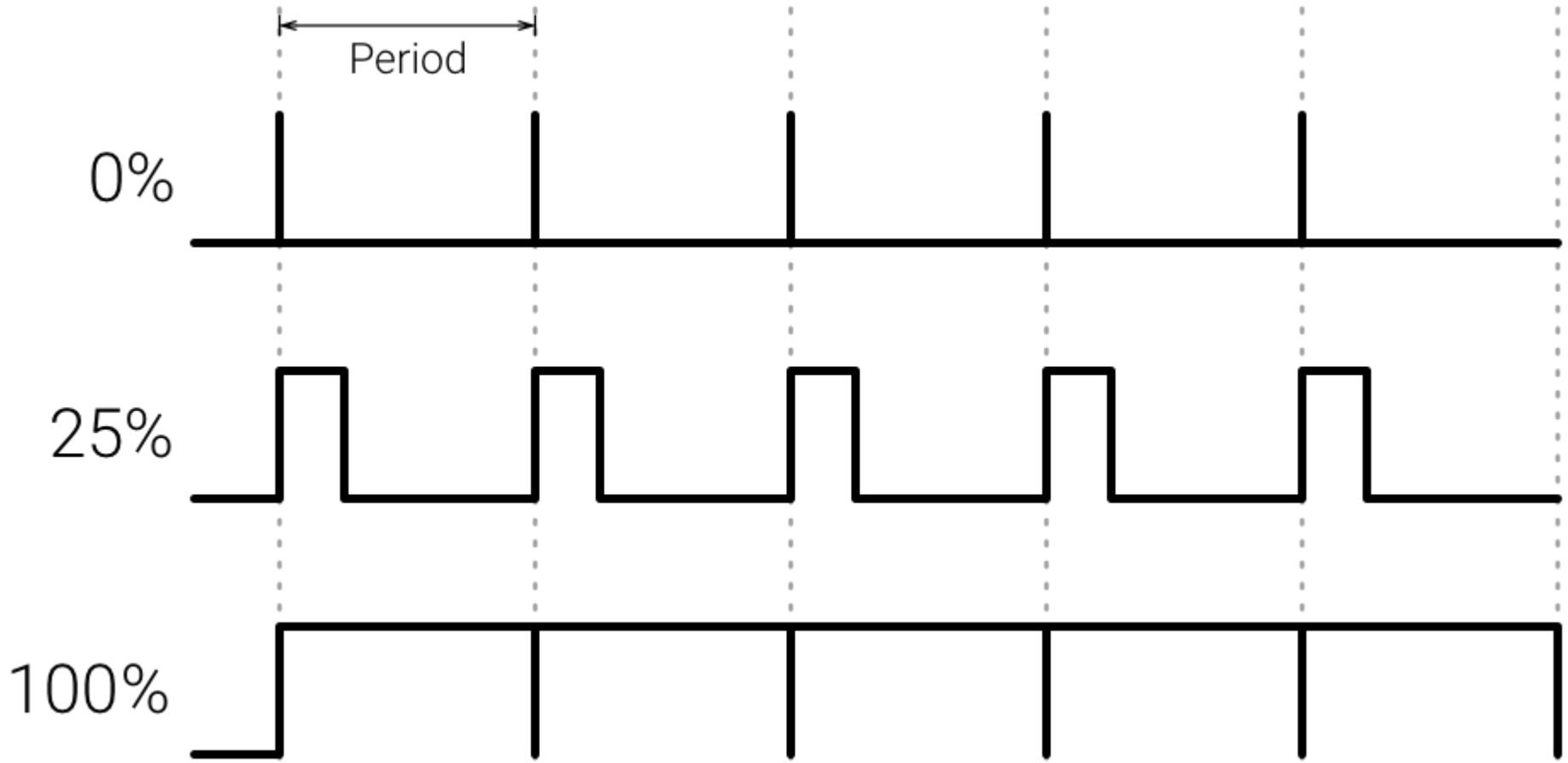
Digital

- Digital is a way of representing voltage in 1 bit: either 0 or 1. Digital pins on the Arduino are pins designed to be configured as inputs or outputs according to the needs of the user. Digital pins are either on or off. When ON they are in a HIGH voltage state of 5V and when OFF they are in a LOW voltage state of 0V.
- On the Arduino, When the digital pins are configured as **output**, they are set to 0 or 5 volts.
- When the digital pins are configured as **input,** the voltage is supplied from an external device. This voltage can vary between 0-5 volts which is converted into digital representation (0 or 1). To determine this, there are 2 thresholds:
- Below 0.8v - considered as 0.
- Above 2v - considered as 1.
- When connecting a component to a digital pin, make sure that the logic levels match. If the voltage is in between the thresholds, the returning value will be undefined.

PWM

In general, Pulse Width Modulation (PWM) is modulation technique used to encode a message into a pulsing signal. A PWM is comprised of two key components: **frequency** and **duty cycle**. The PWM frequency dictates how long it takes to complete a single cycle (period) and how quickly the signal fluctuates from high to low. The duty cycle determines how long a signal stays high out of the total period. Duty cycle is represented in percentage.

PWM signals are used for speed control of DC motors, dimming LEDs and more.


Period
0%
25%
100%

Serial Communication

Serial communication is used to exchange data between the Arduino board and another serial device such as computers, displays, sensors and more. Each Arduino board has at least one serial port. Serial communication occurs on digital pins 0 **(RX)** and 1 **(TX)** as well as via USB.

Arduino supports serial communication through digital pins with the Software Serial Library as well. This allows the user to connect multiple serial-enabled devices and leave the main serial port available for the USB.

SPI

Serial Peripheral Interface (SPI) is a serial data protocol used by microcontrollers to communicate with one or more external devices in a bus like connection. The SPI can also be used to connect 2 microcontrollers.

On the SPI bus, there is always one device that is denoted as a Master device and all the rest as Slaves. In most cases, the microcontroller is the Master device. The SS (Slave Select) pin determines which device the Master is currently communicating with.

Contd.

SPI enabled devices always have the following pins:

- **MISO (Master In Slave Out)** - A line for sending data to the Master device
- **MOSI (Master Out Slave In)** - The Master line for sending data to peripheral devices
- **SCK (Serial Clock)** - A clock signal generated by the Master device to synchronize data transmission.
- **I2C -** SCL/SDA pins are the dedicated pins for I2C communication. On the Arduino Uno they are found on Analog pins A4 and A5.

I2C

- I2C is a communication protocol commonly referred to as the “I2C bus”. The I2C protocol was designed to enable communication between components on a single circuit board. With I2C there are 2 wires referred to as SCL and SDA.
- SCL is the clock line which is designed to synchronize data transfers.
- SDA is the line used to transmit data.
- Each device on the I2C bus has a unique address, up to 255 devices can be connected on the same bus.

Serial Communication Protocols: https://youtu.be/lyGwvGzrqp8

Pin Category	Pin Name	Details
Power	Vin, 3.3V, 5V, GND	Vin: Input voltage to Arduino when using an external power source. 5V: Regulated power supply used to power microcontroller and other components on the board. 3.3V: 3.3V supply generated by on-board voltage regulator. Maximum current draw is 50mA. GND: ground pins.
Reset	Reset	Resets the microcontroller.
Analog Pins	A0 – A5	Used to provide analog input in the range of 0-5V
Input/Output Pins	Digital Pins 0 - 13	Can be used as input or output pins.
Serial	0(Rx), 1(Tx)	Used to receive and transmit TTL serial data.
External Interrupts	2, 3	To trigger an interrupt.
PWM(Pulse Width Modulation)	3, 5, 6, 9, 11	Provides 8-bit PWM output.
SPI (Serial Peripheral interface)	10 (SS), 11 (MOSI), 12 (MISO) and 13 (SCK)	Used for SPI communication. Slave Selection, Master-Slave and serial clock.
Inbuilt LED	13	To turn on the inbuilt LED.
TWI (Two Wire interface)	A4 (SDA), A5 (SCA)	Used for TWI communication.
AREF (Analog REFerence)	AREF	To provide reference voltage for input voltage.

TECH SPECS

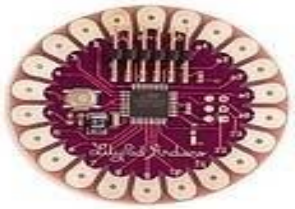
Microcontroller	ATmega328P – 8-bit AVR family microcontroller
Operating Voltage	5V
Recommended Input Voltage	7-12V
Input Voltage Limits	6-20V
Analog Input Pins	6 (A0 – A5)
Digital I/O Pins	14 (Out of which 6 provide PWM output)
DC Current on I/O Pins	40 mA
DC Current on 3.3V Pin	50 mA
Flash Memory	32 KB (0.5 KB is used for Bootloader)
SRAM	2 KB
EEPROM	1 KB
Frequency (Clock Speed)	16 MHz

TYPES OF ARDUINO BOARDS

Arduino Uno

Arduino Leonardo

Arduino Mega 2560



Arduino LilyPad

Arduino Mega ADK

Arduino Nano

Arduino Pro

Arduino Ethernet

Arduino BT

Arduino Fio

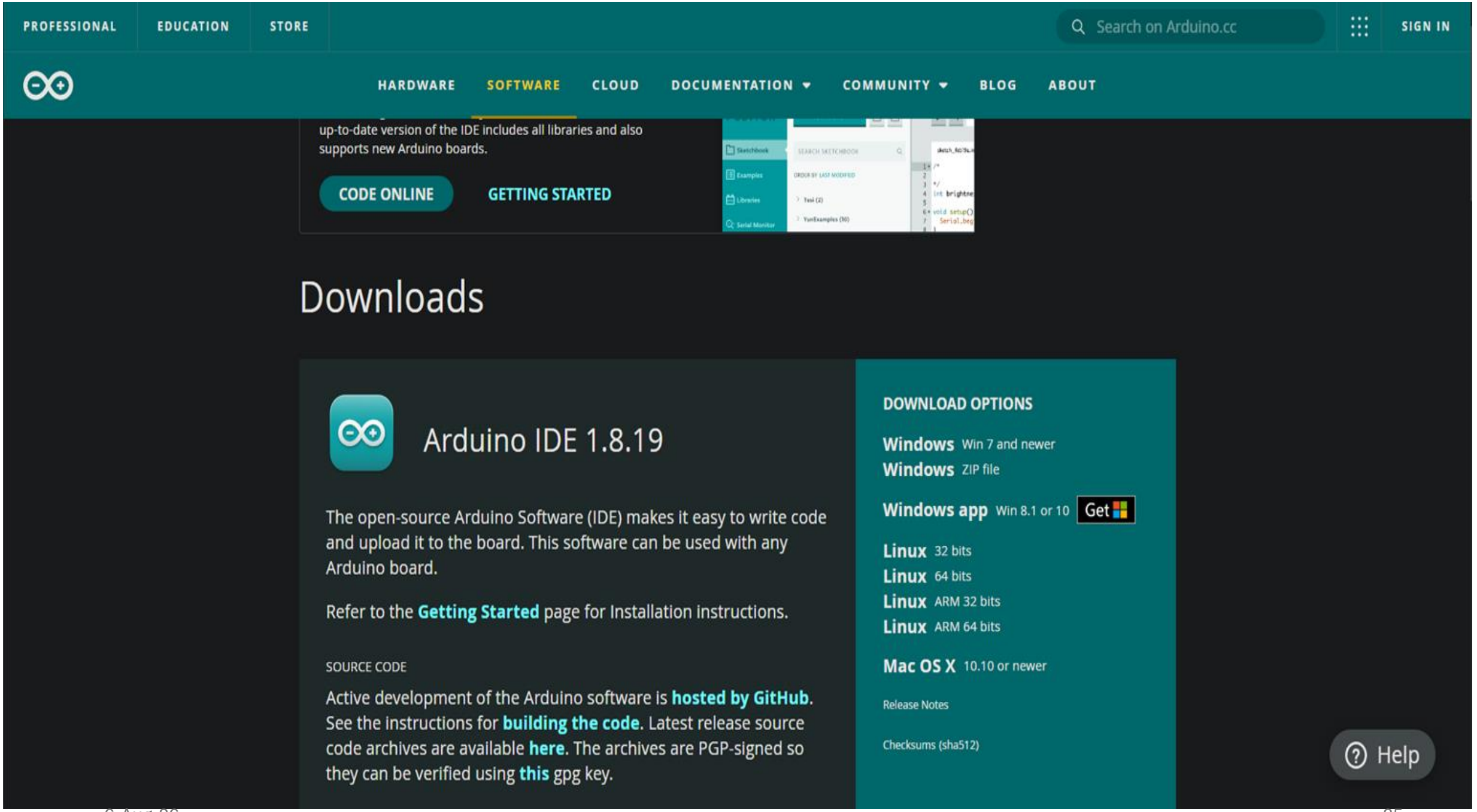
Arduino Mini

Arduino Pro Mini

Types of Arduino Boards

INSTALLATION

- Go to https://www.arduino.cc/en/software
- Download and install Arduino IDE for your OS. (Note: **Do not download the windows app version**, use exe setup).
- You can choose to go for 1.x version if you prefer the legacy else go for 2.x versions

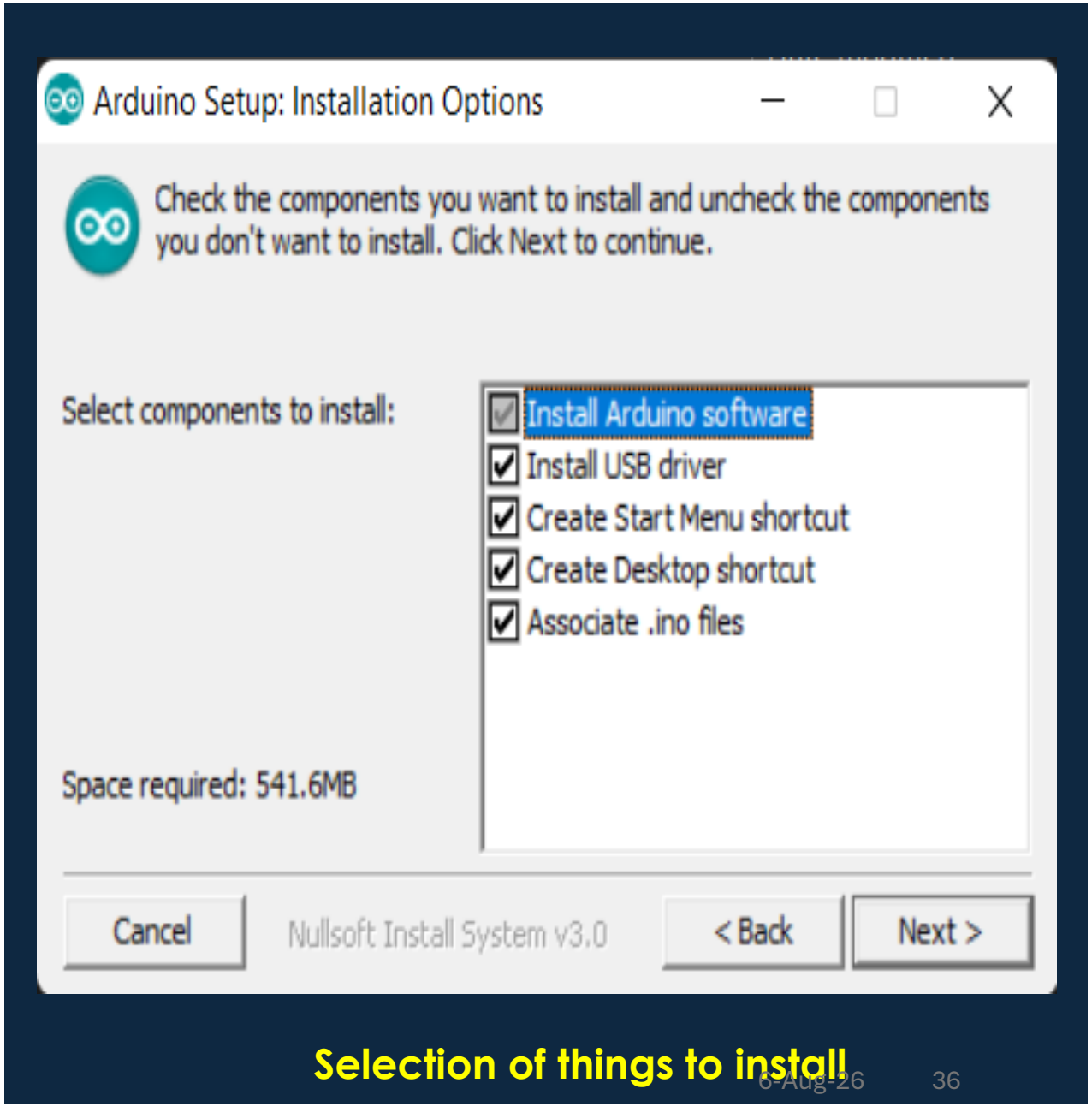


Installation Website

CONTD.

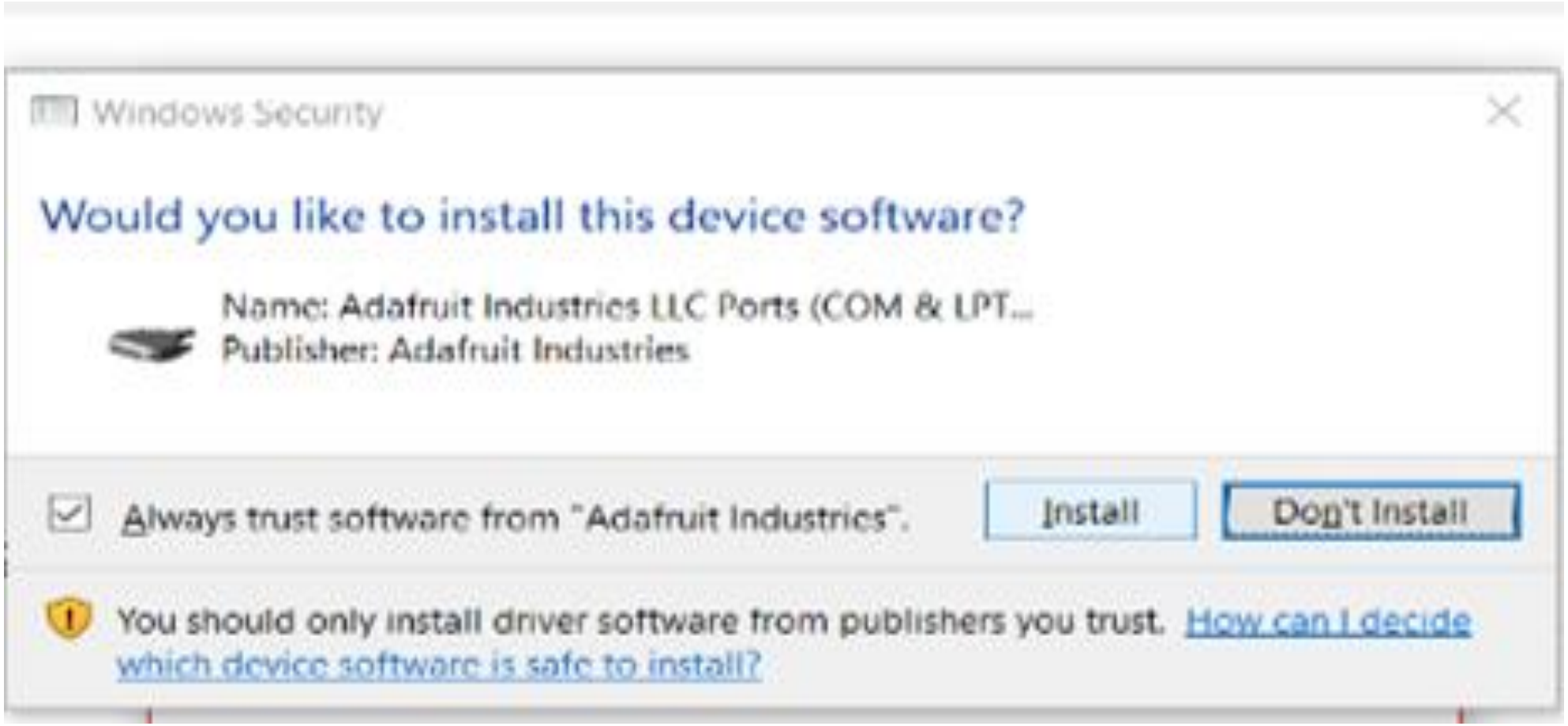
Make sure **“Install USB driver”** is selected.

Click on install to install the drivers



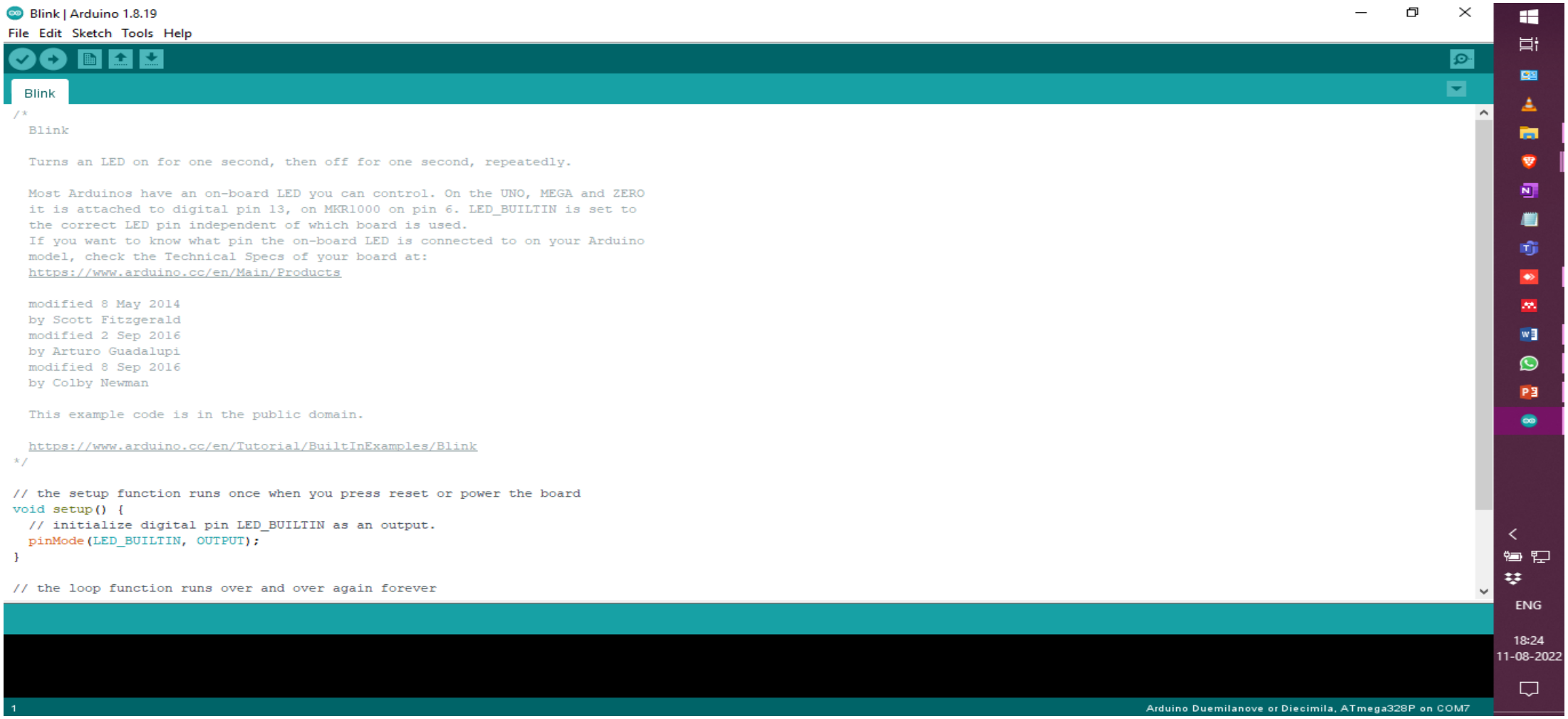
Selection of things to install

CONTD.



Install Driver (Do not skip this)

ARDUINO IDE



View of Arduino 1.x

Python Installation

- Visit the link: **https://www.python.org/downloads/**
- Please download 3.10 or above just to be safe. If you already have it please skip.

CMD Prompt

cd – Change Directory

cd .. – To go back

mkdir – to make directory

dir – To list all directory in the folder (**ls if on linux**)

Environment in Python (without anaconda)

1. **Please open CMD and Navigate to Downloads folder.**
2. **python -m venv iotlab** (This is the command to make a new environment in Python)
3. **iotlab\Scripts\activate** (To activate the environment windows) use **source iotlab/bin/activate** (linux)
4. **Deactivate** (To end the session)
5. **pip install jupyter** (To Install Jupyter Notebook)
6. **jupyter notebook** (To open Notebook) or **jupyter lab** (Lab IDE)

BASICS OF ARDUINO PROGRAMMING

Note: The code snippets are in bold and italic.

STRUCTURE OF THE PROGRAM

Arduino programs can be divided into three main parts:

1. **Structure**
2. **Values** (variables and constants)
3. **Functions**.

STRUCTURE

Software structure consists of two main functions –

1. **Setup( ) function:** The **setup()** function is called when a sketch starts. Use it to initialize the variables, pin modes, start using libraries, etc. The setup function will only run once, after each power up or reset of the Arduino board.
2. **Loop( ) function:** After creating a **setup()** function, which initializes and sets the initial values, the **loop()** function does precisely what its name suggests, and loops consecutively, allowing your program to change and respond. Use it to actively control the Arduino board.

Skeleton

```
void setup() {
  // put your setup code here, to run once:

}

void loop() {
  // put your main code here, to run repeatedly:

}
```

setup()

```
void setup()
{
 pinMode(pin, OUTPUT); // sets the 'pin' as output
}
```

loop()

```
void loop()
{
 digitalWrite(pin, HIGH); // turns 'pin' on
 delay(1000); // pauses for one second
 digitalWrite(pin, LOW); // turns 'pin' off
 delay(1000); // pauses for one second
}
```

Functions

- A function is a block of code that has a name and a block of statements that are executed when the function is called. The functions **void setup()** and **void loop()** have already been discussed and other built-in functions will be discussed later.

type functionName(parameters)

{

statements;

}

Contd.

```
int delayVal()
{
 int v; // create temporary variable 'v'
 v = analogRead(pot); // read potentiometer value
 v /= 4; // converts 0-1023 to 0-255
 return v; // return final value
}
```

Curly Braces{}

Curly braces (also referred to as just "braces" or "curly brackets") define the beginning and end of function blocks and statement blocks such as the void loop() function and the for and if statements.

type function()
{
statements;
}

An opening curly brace { must always be followed by a closing curly brace }. This is often referred to as the braces being balanced. Unbalanced braces can often lead to cryptic, impenetrable compiler errors that can sometimes be hard to track down in a large program.

Semicolan ;

A semicolon must be used to end a statement and separate elements of the program.

A semicolon is also used to separate elements in a for loop.

int x = 13; // declares variable 'x' as the integer 13

Note: Forgetting to end a line in a semicolon will result in a compiler error.

Block Comments /*...*/

Block comments, or multi-line comments, are areas of text ignored by the program and are used for large text descriptions of code or comments that help others understand parts of the program. They begin with /* and end with */ and can span multiple lines.

/* this is an enclosed block comment
don't forget the closing comment -
they have to be balanced!
****/***

Because comments are ignored by the program and take no memory space they should be used generously and can also be used to "comment out" blocks of code for debugging purposes.

Single Line Comment //

Single line comments begin with // and end with the next line of code. Like block comments, they are ignored by the program and take no memory space.

// this is a single line comment

Single line comments are often used after a valid statement to provide more information about what the statement accomplishes or to provide a future reminder.

Variables

A variable is a way of naming and storing a numerical value for later use by the program. As their namesake suggests, variables are numbers that can be continually changed as opposed to constants whose value never changes.

```
int inputVariable = 0;          // declares a variable and
                                // assigns value of 0
inputVariable = analogRead(2);  // set variable to value of
                                // analog pin 2
```

byte

Byte stores an 8-bit numerical value without decimal points. They have a range of 0-255.

```
byte someVariable = 180;          // declares 'someVariable'
                                  // as a byte type
```

int

Integers are the primary datatype for storage of numbers without decimal points and store a 16-bit value with a range of 32,767 to -32,768.

int someVariable = 1500; ***// declares 'someVariable'***

// as an integer type

long

Extended size datatype for long integers, without decimal points, stored in a 32-bit value with a range of 2,147,483,647 to -2,147,483,648.

```
long someVariable = 90000;      // declares 'someVariable'
                                // as a long type
```

float

A datatype for floating-point numbers, or numbers that have a decimal point. Floatingpoint numbers have greater resolution than integers and are stored as a 32-bit value with a range of 3.4028235E+38 to -3.4028235E+38.

float someVariable = 3.14; ***// declares 'someVariable'***

// as a floating-point type

Arrays

An array is a collection of values that are accessed with an index number. Any value in the array may be called upon by calling the name of the array and the index number of the value. Arrays are zero indexed, with the first value in the array beginning at index number 0. An array needs to be declared and optionally assigned values before they can be used.

int myArray[] = {value0, value1, value2...}

Contd.

Likewise, it is possible to declare an array by declaring the array type and size and later assign values to an index position:

int myArray[5]; ***// declares integer array w/ 6 positions***

myArray[3] = 10; ***// assigns the 4th index the value 10***

To retrieve a value from an array, assign a variable to the array and index position:

x = myArray[3]; ***// x now equals 10***

Contd.

```
1.  int ledPin = 10;                                    // LED on pin 10
2.  byte flicker[] = {180, 30, 255, 200, 10, 90, 150, 60};      // above array of 8
3.  void setup()                                    // different values
4.  {
5.   pinMode(ledPin, OUTPUT);                       // sets OUTPUT pin
6.  }
7.  void loop()
8.  {
9.   for(int i=0; i<7; i++)                        // loop equals number
10. {                                               // of values in array
11. analogWrite(ledPin, flicker[i]);                // write index value
12. delay(200);                                     // pause 200ms
13. }}
```

Operators

The following types of operators are available –

1. Arithmetic Operators.
2. Comparison Operators.
3. Boolean Operators.
4. Bitwise Operators.
5. Compound Operators

High/Low

These constants define pin levels as HIGH or LOW and are used when reading or writing to digital pins. HIGH is defined as logic level 1, ON, or 5 volts while LOW is logic level 0, OFF, or 0 volts.

digitalWrite(13, HIGH);

Input/Output

Constants used with the pinMode() function to define the mode of a digital pin as either INPUT or OUTPUT.

pinMode(13, OUTPUT);

Control Statements

IFS

```
if (expression) {
   Block of statements;
}
#####
if (expression) {
   Block of statements;
}
else {
   Block of statements;
}
#####
else if(expression) {
   Block of statements;
}
```

IFS

```
switch (variable) {
   case label:
   // statements
   break;
}

default: {
   // statements
   break;
}
#####
#Conditional
expression1 ? expression2 : expression3
    True         Evaluate        Ignore
    False         Ignore        Evaluate
min = ( a < b ) ? a : b;//To find Min
```

Loops

Loops

```
for ( initialize ;control; increment or decrement) {
   // statement block
   for ( initialize ;control; increment or decrement) {
      // statement block
   }
}
######
#Infinite Loop
#For Loop
for (;;) {
   // statement block
}
#While Loop
while(1) {
   // statement block
}
```

Loops

```
while(expression) {
   Block of statements;
}
######
do {
   Block of statements;
}
while (expression);
######
for ( initialize; control; increment or decrement) {

   // statement block
}
```

pinMode(pin, mode)

Used in void setup() to configure a specified pin to behave either as an INPUT or an OUTPUT.

pinMode(pin, OUTPUT); // sets 'pin' to output

digitalRead(pin) and digitalWrite(pin, value)

Reads the value from a specified digital pin with the result either HIGH or LOW. The pin can be specified as either a variable or constant (0-13).

value = digitalRead(Pin); // sets 'value' equal to the input pin

Outputs either logic level HIGH or LOW at (turns on or off) a specified digital pin. The pin can be specified as either a variable or constant (0-13).

digitalWrite(pin, HIGH); // sets 'pin' to high

analogRead(pin) and analogWrite(pin, value)

Reads the value from a specified analog pin with a 10-bit resolution. This function only works on the analog in pins (0-5). The resulting integer values range from 0 to 1023.

value = analogRead(pin); ***// sets 'value' equal to 'pin'***

analogWrite(pin, value); ***// writes 'value' to analog 'pin'***

delay(ms) and millis()

Delay: Pauses a program for the amount of time as specified in milliseconds, where 1000 equals 1 second.

delay(1000); // waits for one second

Millis: Returns the number of milliseconds since the Arduino board began running the current program as an unsigned long value.

value = millis(); // sets 'value' equal to millis()

Serial.begin(rate)

Opens serial port and sets the baud rate for serial data transmission. The typical baud rate for communicating with the computer is 9600 although other speeds are supported.

```
void setup()
{
 Serial.begin(9600);          // opens serial port
}                             // sets data rate to 9600 bps
```

Serial.println(data)

Prints data to the serial port, followed by an automatic carriage return and line feed. This command takes the same form as Serial.print(), but is easier for reading data on the Serial Monitor.

```
Serial.println(analogValue);          // sends the value of
                                      // 'analogValue'
```

DEMO 1: RUNNING A BASIC CODE

```
void setup()
{
  Serial.begin(9600);
}
void loop()
{
Serial.println(“Hello Welcome \n");
Serial.println("This is Internet of Things.");
delay(1000);
}
```

Run in Linux or Mac Terminal sudo chmod a+rw /dev/ttyACM0 or the port it said

DEMO 2: USING PYTHON FOR SERIAL DATA
(install pyserial and use Jupyter)

```
1. !pip install pyserial
2. import time
3. import serial

4. arduinoData = serial.Serial("COM11", 9600) // Replace COM with yours

5. time.sleep(1)

6. while True:
7.     while(arduinoData.inWaiting()==0):
8.         pass
9.     dataPacket = arduinoData.readline()
10.    print(dataPacket)
```

DEMO 3: BLINKING OF LED ON THE BOARD

DEMO 4: BLINKING OF LED ON THE BOARD WITH USER INPUT

```
int data;
void setup()
{
 Serial.begin(9600);
 pinMode(LED_BUILTIN, OUTPUT);
 digitalWrite (LED_BUILTIN, LOW);                    //initially set to low
 Serial.println("This is my Second Example.");
}
void loop()
{
while (Serial.available())
 {
   data = Serial.read();
 }
 if (data == '1')
 digitalWrite (LED_BUILTIN, HIGH);
 else if (data == '0')
 digitalWrite (LED_BUILTIN, LOW);
}
```

Thanks